1. Exploring Data Types and Type Conversion

Problem Statement: Store a person's name, age (as text), height, and student status in variables, display each variable's data type, and convert the age from text into a whole number to calculate their age next year.

The Math Behind It: Arithmetic addition (age + 1) is only defined for numeric quantities (integers or floating-point numbers), not text strings. Converting a numeric string into an integer allows standard addition to work.

Logic and Algorithm: Create four variables holding a string, a numeric string, a decimal number, and a boolean (True/False) value. Inspect and print each variable's data type. Convert the text age into an integer, add 1 to it, and display the new age.

Pseudocode:

```
EXPLORE-DATA-TYPES()
name ← "Aarav"
age_text ← "20"
height ← 5.9
is_student ← TRUE
print TYPE(name), TYPE(age_text), TYPE(height), TYPE(is_student)
age_number ← INTEGER(age_text)
next_year_age ← age_number + 1
return next_year_age
```

Python Code:

```
name = "Aarav"           # String (str)
age_text = "20"          # Numeric text (str)
height = 5.9             # Decimal number (float)
is_student = True        # Boolean (bool)

print("Type of name:", type(name))
print("Type of age_text:", type(age_text))
print("Type of height:", type(height))
print("Type of is_student:", type(is_student))

age_number = int(age_text)      # Convert str to int
next_year_age = age_number + 1  # Perform integer addition
print("Age next year:", next_year_age)
```

Sample Input and Output:

```
Input: Preset values (name = "Aarav", age_text = "20", height = 5.9,
is_student = True)
Output:
Type of name: <class 'str'>
```

```
Type of age_text: <class 'str'>
Type of height: <class 'float'>
Type of is_student: <class 'bool'>
Age next year: 21
```

Code Explanation: The first four lines assign different data types to variables: text in quotes becomes a str, decimal numbers become a float, and True becomes a bool. The built-in type() function reveals the exact data type of each variable. Finally, int(age_text) converts the text "20" into the integer 20 so Python can add 1 to produce 21.

Execution Trace:

Step	Variables in Memory	Action / Output
1	name="Aarav", age_text="20", height=5.9, is_student=True	Variables initialized with respective types
2	Same as above	Prints str, str, float, and bool
3	age_number = 20 (int)	int("20") converts string to integer
4	next_year_age = 21	Evaluates 20 + 1 and prints Age next year: 21

2. Swap Two Numbers Without a Temporary Variable

Problem Statement: Given two numbers stored in variables a and b, exchange their values without creating a third helper variable.

The Math Behind It: Using addition and subtraction, if we store the combined sum S = a + b in a, then subtracting b from S yields the original a, and subtracting that new b from S yields the original b.

Logic and Algorithm: Take two numbers a and b. Add them together and save the total back into a. Subtract b from this total to recover the original value of a, and store it in b. Finally, subtract the new b from the total in a to recover the original value of b, and store it in a.

Pseudocode:

```
SWAP-NUMBERS(a, b)
print a, b                // display values before swap
a ← a + b                 // a holds combined sum
b ← a - b                 // b gets original value of a
a ← a - b                 // a gets original value of b
return (a, b)
```

Python Code:

```
a = int(input("Enter first number (a): "))
b = int(input("Enter second number (b): "))

print(f"Before swap: a = {a}, b = {b}")

a = a + b  # Step 1: a holds combined sum
b = a - b  # Step 2: b gets original a
a = a - b  # Step 3: a gets original b

print(f"After swap: a = {a}, b = {b}")
```

Sample Input and Output:

```
Input:
Enter first number (a): 15
Enter second number (b): 25

Output:
Before swap: a = 15, b = 25
After swap: a = 25, b = 15
```

Code Explanation: The input() function reads user input as text, and int() converts it into integers. The three arithmetic lines overwrite a with the sum, isolate the original a into b, and isolate the original b into a. Formatted strings (f"...") cleanly print the variables before and after

swapping.

Execution Trace:

Step	Operation	Value of a	Value of b
1	Initial input	15	25
2	a = a + b (15 + 25)	40	25
3	b = a - b (40 - 25)	40	15
4	a = a - b (40 - 15)	25	15

3. Check Whether a Number is Even or Odd

Problem Statement: Read a whole number from the user and determine whether it is even or odd.

The Math Behind It: An integer n is even if dividing it by 2 leaves a remainder of 0 (n mod 2 = 0). If the remainder is not 0, the number is odd.

Logic and Algorithm: Ask the user for an integer. Divide the number by 2 and check the remainder using the modulo operator (%). If the remainder equals 0, report that the number is even; otherwise, report that the number is odd.

Pseudocode:

```
CHECK-EVEN-ODD(num)
if num mod 2 = 0
    return "Even"
else
    return "Odd"
```

Python Code:

```
num = int(input("Enter an integer: "))

if num % 2 == 0:  # Remainder is 0 when divisible by 2
    print(f"{num} is an Even number.")
else:
    print(f"{num} is an Odd number.")
```

Sample Input and Output:

```
Input:
Enter an integer: 14

Output:
14 is an Even number.
```

Code Explanation: The % operator computes the remainder of integer division. The if statement checks whether num % 2 == 0 is true. If true, the indented block under if runs; otherwise, the indented block under else runs.

Execution Trace:

Step	Expression / State	Result
1	User input	num = 14
2	14 % 2	0

Step	Expression / State	Result
3	Condition 0 == 0	True → prints 14 is an Even number.

4. Find the Largest of Three Numbers

Problem Statement: Accept three numbers from the user and determine which one is the largest.

The Math Behind It: By order comparison, a value a is the maximum of {a, b, c} if and only if a ≥ b and a ≥ c.

Logic and Algorithm: Read three numbers a, b, and c. First check if a is greater than or equal to both b and c. If not, check if b is greater than or equal to both a and c. If neither is true, then c must be the largest.

Pseudocode:

```
LARGEST-OF-THREE(a, b, c)
if a ≥ b and a ≥ c
    largest ← a
elseif b ≥ a and b ≥ c
    largest ← b
else
    largest ← c
return largest
```

Python Code:

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a >= b and a >= c:    # Test if a is largest
    largest = a
elif b >= a and b >= c:  # Test if b is largest
    largest = b
else:                    # Otherwise c must be largest
    largest = c

print("The largest number is:", largest)
```

Sample Input and Output:

```
Input:
Enter first number: 12
Enter second number: 45
Enter third number: 29

Output:
The largest number is: 45.0
```

Code Explanation: Using float() allows both integers and decimals to be compared. The logical operator and requires both comparisons on its left and right to be true. The if-elif-else chain tests each candidate in order and stores the winning number in largest.

Execution Trace:

Step	Condition Tested	Outcome
1	Inputs stored	a = 12.0, b = 45.0, c = 29.0
2	12.0 >= 45.0 and 12.0 >= 29.0	False → moves to elif
3	45.0 >= 12.0 and 45.0 >= 29.0	True → largest = 45.0, prints 45.0

5. Calculate Simple Interest

Problem Statement: Given a principal amount, an annual interest rate in percent, and a time duration in years, calculate the simple interest earned and the final total amount.
The Math Behind It: Simple Interest is calculated as SI = (P × R × T) / 100, where P is principal, R is the annual interest rate, and T is time in years. Total amount is P + SI.
Logic and Algorithm: Read the principal, rate, and time values. Multiply all three numbers together and divide by 100 to obtain the simple interest. Add the simple interest to the original principal to find the total amount, then print both results.
Pseudocode:

```
SIMPLE-INTEREST(principal, rate, time)
interest ← (principal * rate * time) / 100
total ← principal + interest
return (interest, total)
```

Python Code:

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter annual interest rate (%): "))
time = float(input("Enter time in years: "))

interest = (principal * rate * time) / 100
total_amount = principal + interest

print("Simple Interest:", interest)
print("Total Amount:", total_amount)
```

Sample Input and Output:

```
Input:
Enter principal amount: 5000
Enter annual interest rate (%): 6
Enter time in years: 3

Output:
Simple Interest: 900.0
Total Amount: 5900.0
```

Code Explanation: The three inputs are converted to float so fractional rates or years work properly. Parentheses ensure principal * rate * time is multiplied completely before dividing by 100. Both the computed interest and total balance are then displayed.
Execution Trace:

Step	Calculation	Stored Value
1	Read inputs	principal=5000.0, rate=6.0, time=3.0
2	(5000.0 * 6.0 * 3.0) / 100	interest = 900.0
3	5000.0 + 900.0	total_amount = 5900.0

6. Convert Celsius to Fahrenheit

Problem Statement: Convert a temperature given in degrees Celsius into degrees Fahrenheit.
The Math Behind It: The linear conversion formula is F = (C × 9/5) + 32, accounting for the 32-degree freezing point offset and the 1.8x degree scale difference.
Logic and Algorithm: Take a Celsius temperature from the user. Multiply it by 9, divide by 5, and add 32 to the result. Print the converted Fahrenheit temperature.
Pseudocode:

```
CELSIUS-TO-FAHRENHEIT(celsius)
fahrenheit ← (celsius * 9 / 5) + 32
return fahrenheit
```

Python Code:

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
print(f"{celsius}°C is equal to {fahrenheit}°F")
```

Sample Input and Output:

```
Input:
Enter temperature in Celsius: 25

Output:
25.0°C is equal to 77.0°F
```

Code Explanation: The input is read as a float to support decimal temperatures. Python evaluates celsius * 9 / 5 from left to right before adding 32, then formats both values into a readable sentence.
Execution Trace:

Step	Expression	Value
1	Input read	celsius = 25.0
2	25.0 * 9 / 5	45.0
3	45.0 + 32	fahrenheit = 77.0 → prints 25.0°C is equal to 77.0°F

7. Sum of First N Natural Numbers

Problem Statement: Given a positive integer N, compute the sum of all natural numbers from 1 up to N.
The Math Behind It: The cumulative sum is 1 + 2 + 3 + ... + N (which also equals N(N + 1) / 2).
Logic and Algorithm: Start with a running total equal to 0. Loop through every integer from 1 to N, adding each number to the running total. Once the loop ends, print the final total.
Pseudocode:

```
SUM-NATURAL-NUMBERS(n)
total ← 0
for i ← 1 to n
    total ← total + i
return total
```

Python Code:

```
n = int(input("Enter a positive integer (N): "))
total = 0

for i in range(1, n + 1):  # Loops from 1 through n inclusive
    total += i             # Add current i to running total

print(f"The sum of the first {n} natural numbers is: {total}")
```

Sample Input and Output:

```
Input:
Enter a positive integer (N): 5

Output:
The sum of the first 5 natural numbers is: 15
```

Code Explanation: range(1, n + 1) generates numbers starting at 1 and stopping at n (since Python's range excludes the upper bound). The shorthand total += i adds the current value of i to total on every pass of the loop.
Execution Trace:

Iteration	i	Calculation (total + i)	New total
Start	-	Initial state (n = 5)	0
1	1	0 + 1	1
2	2	1 + 2	3

Iteration	i	Calculation (total + i)	New total
3	3	3 + 3	6
4	4	6 + 4	10
5	5	10 + 5	15 (Final)

8. Factorial of a Number

Problem Statement: Calculate the factorial of a non-negative integer N.
The Math Behind It: The factorial of N, written N!, is the product of all positive integers up to N: N! = 1 × 2 × 3 × ... × N, with 0! = 1.
Logic and Algorithm: Initialize a result variable factorial to 1. Loop through every integer from 1 to N and multiply factorial by each integer. After the loop completes, output the accumulated product.
Pseudocode:

```
FACTORIAL(n)
factorial ← 1
for i ← 1 to n
    factorial ← factorial * i
return factorial
```

Python Code:

```
n = int(input("Enter a non-negative integer: "))
factorial = 1

for i in range(1, n + 1):
    factorial *= i  # Multiply running product by i

print(f"The factorial of {n} is: {factorial}")
```

Sample Input and Output:

```
Input:
Enter a non-negative integer: 5

Output:
The factorial of 5 is: 120
```

Code Explanation: Starting factorial at 1 (the multiplicative identity) ensures multiplication works properly and naturally handles 0! = 1 because the loop runs zero times when n = 0. The operator *= multiplies factorial by i in each step.
Execution Trace:

Iteration	i	Calculation (factorial * i)	New factorial
Start	-	Initial state (n = 5)	1
1	1	1 * 1	1

Iteration	i	Calculation (factorial * i)	New factorial
2	2	1 * 2	2
3	3	2 * 3	6
4	4	6 * 4	24
5	5	24 * 5	120 (Final)

9. Generate the Fibonacci Sequence up to N Terms

Problem Statement: Print the first N terms of the Fibonacci sequence.
The Math Behind It: The sequence starts with $F_0 = 0$ and $F_1 = 1$, and every subsequent term is the sum of the previous two: $F_k = F_{k-1} + F_{k-2}$.
Logic and Algorithm: Keep track of two variables, a = 0 and b = 1. Repeat N times: print the current value of a, compute the next term a + b, shift b into a, and put the new sum into b.
Pseudocode:

```
FIBONACCI-SEQUENCE(n)
a ← 0
b ← 1
for count ← 1 to n
    print a
    next_term ← a + b
    a ← b
    b ← next_term
```

Python Code:

```
n = int(input("Enter number of terms: "))
a, b = 0, 1

print("Fibonacci sequence:", end=" ")
for _ in range(n):
    print(a, end=" ")     # Print current term
    next_term = a + b     # Compute next term
    a = b                 # Shift b into a
    b = next_term         # Update b with new sum
```

Sample Input and Output:

```
Input:
Enter number of terms: 6

Output:
Fibonacci sequence: 0 1 1 2 3 5
```

Code Explanation: The underscore _ is used as a loop variable when we only care about repeating the loop n times. Passing end=" " to print() keeps all numbers on the same line separated by spaces. Inside the loop, a and b slide forward by one position each iteration.
Execution Trace:

Step	Printed (a)	next_term (a + b)	Updated (a, b)
1	0	0 + 1 = 1	a = 1, b = 1
2	1	1 + 1 = 2	a = 1, b = 2
3	1	1 + 2 = 3	a = 2, b = 3
4	2	2 + 3 = 5	a = 3, b = 5
5	3	3 + 5 = 8	a = 5, b = 8
6	5	5 + 8 = 13	a = 8, b = 13 (End)

10. Reverse a Given Integer

Problem Statement: Take a positive integer and produce a new integer with its digits in reverse order (for example, turning 1234 into 4321).

The Math Behind It: In base 10, n mod 10 extracts the rightmost digit, while integer division ⌊n / 10⌋ drops the rightmost digit. Multiplying the reversed accumulator by 10 shifts its existing digits left so the extracted digit can be added at the ones place.

Logic and Algorithm: Set reversed_num = 0. While the input number is greater than 0, grab its last digit using % 10, append it to reversed_num by doing reversed_num * 10 + digit, and chop off the last digit of the input number using integer division // 10.

Pseudocode:

```
REVERSE-INTEGER(num)
reversed_num ← 0
while num > 0
    digit ← num mod 10                    // extract rightmost digit
    reversed_num ← (reversed_num * 10) + digit
    num ← ⌊num / 10⌋                      // drop rightmost digit
return reversed_num
```

Python Code:

```
num = int(input("Enter a positive integer: "))
reversed_num = 0

while num > 0:
    digit = num % 10        # Extract last digit
    reversed_num = reversed_num * 10 + digit
    num = num // 10         # Remove last digit

print("Reversed number:", reversed_num)
```

Sample Input and Output:

```
Input:
Enter a positive integer: 1234

Output:
Reversed number: 4321
```

Code Explanation: The while loop continues as long as digits remain in num. The floor division operator // divides by 10 and discards the decimal remainder, shrinking num by one digit per iteration until it reaches 0.

Execution Trace:

Pass	digit (num % 10)	reversed_num	Remaining num (num // 10)
Start	-	0	1234
1	4	0 * 10 + 4 = 4	123
2	3	4 * 10 + 3 = 43	12
3	2	43 * 10 + 2 = 432	1
4	1	432 * 10 + 1 = 4321	0 (Loop stops)

11. Check Whether a Number is a Palindrome

Problem Statement: Determine whether a given positive integer reads the exact same backward as forward (such as 121 or 1331).

The Math Behind It: A number x is a palindrome if reversing its base-10 digits produces an integer y such that x = y.

Logic and Algorithm: Save a copy of the original number in a variable called original. Reverse the digits of a working copy using the digit-extraction loop. Compare the reversed result with original: if they match, it is a palindrome.

Pseudocode:

```
IS-PALINDROME(num)
original ← num
reversed_num ← 0
while num > 0
    digit ← num mod 10
    reversed_num ← (reversed_num * 10) + digit
    num ← ⌊num / 10⌋
if original = reversed_num
    return TRUE
else
    return FALSE
```

Python Code:

```
num = int(input("Enter a positive integer: "))
original = num      # Preserve original value
reversed_num = 0

while num > 0:
    digit = num % 10
    reversed_num = reversed_num * 10 + digit
    num = num // 10

if original == reversed_num:
    print(f"{original} is a Palindrome number.")
else:
    print(f"{original} is not a Palindrome number.")
```

Sample Input and Output:

```
Input:
Enter a positive integer: 121

Output:
```

```
121 is a Palindrome number.
```

Code Explanation: Because the while loop reduces num down to 0, storing original = num beforehand preserves the initial value so we can compare it against reversed_num at the end.
Execution Trace:

Pass	digit	reversed_num	num	original
Start	-	0	121	121
1	1	1	12	121
2	2	12	1	121
3	1	121	0	121 (Match: True)

12. Find the Sum of Digits of a Number

Problem Statement: Calculate the sum of all individual digits in a given positive integer.
The Math Behind It: Any positive integer can be decomposed digit-by-digit using modulo 10 (n mod 10) to isolate the last digit and integer division (⌊n / 10⌋) to shift to the next digit.
Logic and Algorithm: Start with digit_sum = 0. Repeatedly extract the rightmost digit of the number with % 10, add it to digit_sum, and remove that digit from the number using // 10 until the number becomes 0.
Pseudocode:

```
SUM-OF-DIGITS(num)
digit_sum ← 0
while num > 0
    digit_sum ← digit_sum + (num mod 10)
    num ← ⌊num / 10⌋
return digit_sum
```

Python Code:

```
num = int(input("Enter a positive integer: "))
digit_sum = 0

while num > 0:
    digit_sum += num % 10   # Add last digit to sum
    num //= 10              # Drop last digit

print("Sum of digits:", digit_sum)
```

Sample Input and Output:

```
Input:
Enter a positive integer: 472

Output:
Sum of digits: 13
```

Code Explanation: digit_sum += num % 10 combines extracting the last digit and adding it to the accumulator in a single line. num //= 10 updates num in place by dropping the digit just processed.
Execution Trace:

Pass	Extracted Digit (num % 10)	digit_sum	Remaining num (num // 10)
Start	-	0	472

Pass	Extracted Digit (num % 10)	digit_sum	Remaining num (num // 10)
1	2	0 + 2 = 2	47
2	7	2 + 7 = 9	4
3	4	9 + 4 = 13	0 (Final: 13)

13. Check Whether a Number is Prime

Problem Statement: Determine whether a given integer is a prime number (an integer greater than 1 with no positive divisors other than 1 and itself).

The Math Behind It: If a composite number n has a factor larger than √n, its paired factor must be smaller than √n. Therefore, we only need to test divisibility (n mod i = 0) for integers i from 2 up to ⌊√n⌋.

Logic and Algorithm: If the number is less than or equal to 1, it is not prime. Otherwise, assume it is prime (is_prime = True) and test dividing it by every integer from 2 up to its square root. If any divisor leaves a remainder of 0, mark is_prime = False and stop early.

Pseudocode:

```
IS-PRIME(num)
if num ≤ 1
    return FALSE
for i ← 2 to ⌊√num⌋
    if num mod i = 0
        return FALSE         // found a non-trivial divisor
return TRUE
```

Python Code:

```
num = int(input("Enter an integer: "))

if num <= 1:
    is_prime = False
else:
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:    # Found a divisor other than 1 and num
            is_prime = False
            break

if is_prime:
    print(f"{num} is a Prime number.")
else:
    print(f"{num} is not a Prime number.")
```

Sample Input and Output:

```
Input:
Enter an integer: 17

Output:
```

```
17 is a Prime number.
```

Code Explanation: Raising num ** 0.5 computes the square root of num, and int(...) rounds it down to a whole number. The break keyword immediately exits the loop as soon as a factor is found, saving unnecessary checks.

Execution Trace:

Divisor (i)	Remainder (17 % i)	is_prime	Action
Start	-	True	int(17 ** 0.5) = 4; test i = 2..4
2	17 % 2 = 1	True	Not divisible; continue
3	17 % 3 = 2	True	Not divisible; continue
4	17 % 4 = 1	True	Loop ends; prints 17 is a Prime number.

14. Find the Greatest Common Divisor (GCD) of Two Numbers

Problem Statement: Find the largest positive integer that divides two given positive integers without leaving any remainder.

The Math Behind It: By the Euclidean algorithm, gcd(a, b) = gcd(b, a mod b). Repeating this reduction eventually produces a remainder of 0, at which point the non-zero value is the GCD.

Logic and Algorithm: Take two positive integers a and b. While b is not 0, compute the remainder of a / b, set a equal to b, and set b equal to that remainder. Once b becomes 0, a holds the GCD.

Pseudocode:

```
EUCLID-GCD(a, b)
while b ≠ 0
    remainder ← a mod b
    a ← b
    b ← remainder
return a
```

Python Code:

```
a = int(input("Enter first positive integer: "))
b = int(input("Enter second positive integer: "))

while b != 0:
    remainder = a % b   # Remainder of a divided by b
    a = b               # Shift b into a
    b = remainder       # Shift remainder into b

print("The GCD is:", a)
```

Sample Input and Output:

```
Input:
Enter first positive integer: 48
Enter second positive integer: 18

Output:
The GCD is: 6
```

Code Explanation: The != operator checks "not equal to". Storing a % b in remainder before overwriting a ensures we do not lose the value needed to update b. Even if a < b initially, the first iteration automatically swaps them.

Execution Trace:

Pass	Values (a, b)	Remainder (a % b)	Updated State
1	a = 48, b = 18	48 % 18 = 12	a = 18, b = 12
2	a = 18, b = 12	18 % 12 = 6	a = 12, b = 6
3	a = 12, b = 6	12 % 6 = 0	a = 6, b = 0 → prints 6

15. Count the Number of Vowels in a String

Problem Statement: Count how many vowel letters (a, e, i, o, u, case-insensitive) appear in a given word or sentence.

The Math Behind It: Set membership counting: given a sequence of characters, increment a counter by 1 for each character c that belongs to the vowel set V = {a, e, i, o, u, A, E, I, O, U}.

Logic and Algorithm: Define a string containing all uppercase and lowercase vowels. Initialize count = 0. Examine every character in the input text one by one; if the character appears in the vowel string, add 1 to count.

Pseudocode:

```
COUNT-VOWELS(T, n)
// T[1 .. n] is an array of characters
count ← 0
for i ← 1 to n
    if T[i] ∈ {'a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'}
        count ← count + 1
return count
```

Python Code:

```
text = input("Enter a word or sentence: ")
vowels = "aeiouAEIOU"
count = 0

for char in text:
    if char in vowels:  # Check if character is a vowel
        count += 1

print("Number of vowels:", count)
```

Sample Input and Output:

```
Input:
Enter a word or sentence: Python Programming

Output:
Number of vowels: 4
```

Code Explanation: In Python, strings are iterable, so for char in text: visits each character from left to right. The in operator checks whether char matches any character inside vowels.

Execution Trace:

Segment Scanned	Matched Vowel	Updated count
"Pyth"	None	0
"o"	'o'	1
"n Pro"	'o'	2
"gra"	'a'	3
"mming"	'i'	4 (Final)

16. Find the Largest and Smallest Element in a List

Problem Statement: Given a list of numbers, find both the smallest and largest values without using Python's built-in min() or max() functions.

The Math Behind It: Sequential extrema tracking: start by assuming the first element is both the minimum and maximum, then update the minimum whenever x < min_val and update the maximum whenever x > max_val.

Logic and Algorithm: Set both smallest and largest to the first item in the list. Loop through every number in the list: if a number is smaller than smallest, replace smallest with it; if it is larger than largest, replace largest with it.

Pseudocode:

```
FIND-MIN-MAX(A, n)
// A[1 .. n] is a non-empty array of numbers
smallest ← A[1]
largest ← A[1]
for i ← 2 to n
    if A[i] < smallest
        smallest ← A[i]
    if A[i] > largest
        largest ← A[i]
return (smallest, largest)
```

Python Code:

```
numbers = [18, 7, 42, 3, 29]
smallest = numbers[0]   # Initialize with first element
largest = numbers[0]

for num in numbers:
    if num < smallest:
        smallest = num
    if num > largest:
        largest = num

print("Smallest number:", smallest)
print("Largest number:", largest)
```

Sample Input and Output:

```
Input: Preset list [18, 7, 42, 3, 29]
Output:
Smallest number: 3
Largest number: 42
```

Code Explanation: Lists in Python are zero-indexed, meaning numbers[0] accesses the first element (18), whereas CLRS pseudocode uses 1-based indexing (A[1]). Initializing with the first element rather than 0 guarantees the logic works even if all numbers in the list are negative.

Execution Trace:

num	smallest	largest	Action
18	18	18	No change
7	7	18	7 < 18 → updates smallest
42	7	42	42 > 18 → updates largest
3	3	42	3 < 7 → updates smallest
29	3	42	No change (Final: 3 and 42)

17. Find an Item in a List (Linear Search)

Problem Statement: Search for a specific target number inside a list and print its index position if found, or report that it does not exist in the list.

The Math Behind It: Linear search checks elements sequentially across positions 1..n (or 0..n - 1 in Python) until it locates the first position where the element equals target.

Logic and Algorithm: Set found_index = -1 (a sentinel value indicating "not found"). Loop through all valid indices of the list. If the item at index i equals the target, record found_index = i and stop searching. Afterward, check if found_index is still -1.

Pseudocode:

```
LINEAR-SEARCH(A, n, target)
// A[1 .. n] is an array of n elements
for i ← 1 to n
    if A[i] = target
        return i             // 1-based position where target is found
return NIL                   // target not found in A[1 .. n]
```

Python Code:

```
numbers = [10, 25, 30, 45, 50]
target = int(input("Enter number to search: "))
found_index = -1    # Sentinel value for "not found"

for i in range(len(numbers)):
    if numbers[i] == target:
        found_index = i
        break       # Stop once target is located

if found_index != -1:
    print(f"{target} found at index {found_index}.")
else:
    print(f"{target} was not found in the list.")
```

Sample Input and Output:

```
Input:
List: [10, 25, 30, 45, 50]
Enter number to search: 30

Output:
30 found at index 2.
```

Code Explanation: len(numbers) returns 5, so range(5) generates 0-based Python indices 0, 1, 2, 3, 4 (corresponding to 1-based positions 1..5 in CLRS pseudocode). Using -1 as the default

sentinel value in Python is standard because valid list indices are always 0 or greater.

Execution Trace:

Index (i)	numbers[i]	Comparison (== 30)	Action / found_index
0	10	False	Continue (found_index = -1)
1	25	False	Continue (found_index = -1)
2	30	True	found_index = 2, break → prints index 2

18. Reverse a List Without Built-in Reverse

Problem Statement: Reverse the order of items inside a list in place without using .reverse() or list slicing ([::-1]).

The Math Behind It: Symmetric index reflection: in a sequence of length n, the element at left swaps with the element at right = n + 1 - left (1-based) while left < right.

Logic and Algorithm: Place one pointer (left) at the beginning of the list and another pointer (right) at the end of the list. While left is less than right, swap the elements at those two positions, then move left forward by 1 and right backward by 1.

Pseudocode:

```
REVERSE-ARRAY(A, n)
// Reverses A[1 .. n] in place
left ← 1
right ← n
while left < right
    exchange A[left] with A[right]
    left ← left + 1
    right ← right - 1
return A
```

Python Code:

```
items = [10, 20, 30, 40, 50]
left = 0
right = len(items) - 1

while left < right:
    items[left], items[right] = items[right], items[left]  # Swap elements
    left += 1
    right -= 1

print("Reversed list:", items)
```

Sample Input and Output:

```
Input: Preset list [10, 20, 30, 40, 50]
Output:
Reversed list: [50, 40, 30, 20, 10]
```

Code Explanation: Python's tuple unpacking items[left], items[right] = items[right], items[left] swaps two list elements in place without a temporary variable (matching CLRS's exchange A[left] with A[right]). The loop stops once the pointers meet or cross in the middle.

Execution Trace:

Pass	Pointers (left, right)	Swapped Elements	List State
Start	(0, 4)	-	[10, 20, 30, 40, 50]
1	(0, 4) → (1, 3)	10 ↔ 50	[50, 20, 30, 40, 10]
2	(1, 3) → (2, 2)	20 ↔ 40	[50, 40, 30, 20, 10] (Loop stops)

19. Sort a List in Ascending Order (Bubble Sort)

Problem Statement: Sort a list of numbers into ascending order (smallest to largest) without using Python's built-in .sort() or sorted() functions.

The Math Behind It: Adjacent pairwise comparison: if A[j] > A[j + 1], swapping them eliminates an inversion. After i passes through an n-element array, the largest i elements are guaranteed to be in their final sorted positions at the end.

Logic and Algorithm: Run an outer loop n - 1 times. In each pass, run an inner loop comparing each element with its immediate right neighbor; if the left element is greater than the right element, swap them.

Pseudocode:

```
BUBBLE-SORT(A, n)
// Sorts A[1 .. n] into ascending order
for i ← 1 to n - 1
    for j ← 1 to n - i
        if A[j] > A[j + 1]
            exchange A[j] with A[j + 1]
return A
```

Python Code:

```
nums = [64, 34, 25, 12, 22]
n = len(nums)

for i in range(n - 1):
    for j in range(n - 1 - i):
        if nums[j] > nums[j + 1]:
            nums[j], nums[j + 1] = nums[j + 1], nums[j]

print("Sorted list:", nums)
```

Sample Input and Output:

```
Input: Preset list [64, 34, 25, 12, 22]
Output:
Sorted list: [12, 22, 25, 34, 64]
```

Code Explanation: The outer loop controls the number of passes across the list. With each completed pass, the next largest value "bubbles up" to its proper spot at the end, so the inner loop shrinks its range by i.

Execution Trace:

Pass	Largest Element Placed	List State After Pass
Start	-	[64, 34, 25, 12, 22]
1	64	[34, 25, 12, 22, 64]
2	34	[25, 12, 22, 34, 64]
3	25	[12, 22, 25, 34, 64]
4	22 (All sorted)	[12, 22, 25, 34, 64]

20. Count How Many Times an Item Appears in a List

Problem Statement: Given a list of items and a target value, count how many times the target value occurs in the list without using .count().

The Math Behind It: Frequency counting (cardinality of matching indices): count = |{i : A[i] = target}|.

Logic and Algorithm: Set a counter variable count = 0. Visit every item in the list; whenever an item equals the target value, increment count by 1. Print count after checking the entire list.

Pseudocode:

```
COUNT-OCCURRENCES(A, n, target)
// A[1 .. n] is an array of n elements
count ← 0
for i ← 1 to n
    if A[i] = target
        count ← count + 1
return count
```

Python Code:

```
items = [4, 2, 7, 2, 9, 2, 5]
target = 2
count = 0

for item in items:
    if item == target:
        count += 1

print(f"{target} appears {count} times in the list.")
```

Sample Input and Output:

```
Input: Preset list [4, 2, 7, 2, 9, 2, 5], target = 2
Output:
2 appears 3 times in the list.
```

Code Explanation: The for loop iterates over every element in items. Each time item == target evaluates to True, count increases by 1. At the end of the loop, count holds the total frequency of target.

Execution Trace:

Index	item	item == 2?	count
0	4	False	0

Index	item	item == 2?	count
1	2	True	1
2	7	False	1
3	2	True	2
4	9	False	2
5	2	True	3
6	5	False	3 (Final)